Newton

Inovação é a nossa tradição.

Newton
Inovação é a nossa tradição.

Estruturas de Dados

Professor Fabrício Rodrigues Inácio
fabricio.inacio@professores.newtonpaiva.edu.br

Estruturas de Dados

- O que é uma estrutura de dados?
- Por que precisamos usar estruturas de dados?
- Exemplos de estruturas de dados?

Estruturas de Dados

Os tipos de dados definidos por uma linguagem de programação não são suficientes para modelar as diversas situações do ambiente dentro do código de programação, nem tudo pode ser representado como um valor inteiro int ou um valor real float ou ainda um caracter char sendo que no caso da linguagem C, não conseguimos representar nem mesmo uma palavra (conjunto de caracteres chamado de String em seus tipos primitivos, sendo necessária a declaração de um vetor para armazenar uma cadeia de caracteres char formando uma String.

Estruturas de Dados

A criação de novos tipos de dados, denominados tipos construídos, elaborados a partir dos tipos primitivos consegue solucionar essa limitação e destaca que “esses novos tipos têm um formato denominado estrutura de dados, que define como os tipos primitivos estão organizados” Forbellone (2005).

Estruturas de Dados – Aplicação

- **Persistência temporária de dados**

Os programas de forma geral precisam acessar recursos externos ao programa, como bancos de dados e informações de computadores e servidores locais ou através da Internet, a recuperação de dados externos à memória RAM é um limitador na velocidade do programa.

Se todas as vezes que uma informação externa for necessária, o programa tiver que buscá-la, ele perderá muito tempo. No contexto apresentado, as estruturas de dados vêm para possibilitar que seja feita uma persistência temporária de dados.

Estruturas de Dados – Aplicação

- **Processamento digital de imagens**

A amostragem consiste em discretizar o domínio de definição da imagem nas direções x e y, gerando uma matriz de M x N amostras, respectivamente. Cada elemento f(x, y) dessa matriz de amostras é chamado pixel (acrônimo do inglês Picture element).

Para as imagens coloridas no padrão RGB, Pedrini (2008, p. 471) destaca que passa a ser necessária uma matriz com 3 dimensões, sendo a terceira dimensão responsável pela cor do pixel que pode variar os valores de R, G e B entre valores de 0 a 255 para cada cor sendo um pixel capaz de formar mais de 16 milhões de cores.

Vetores

Um vetor é um conjunto de variáveis de mesmo tipo que podem ser referenciadas por um único identificador, chamamos de vetores as matrizes de uma dimensão, conforme define Ascencio (2005, p.1) “Um vetor é também chamado de variável composta homogênea unidimensional, ou seja, uma sequência finita de variáveis todas do mesmo tipo, com o mesmo identificador (mesmo nome) e alocadas sequencialmente na memória.

a	d	R	@	t	s	G	%	ç	p

Vetores

Um vetor é um conjunto de variáveis de mesmo tipo que podem ser referenciadas por um único identificador, chamamos de vetores as matrizes de uma dimensão, conforme define Ascencio (2005, p.1) “Um vetor é também chamado de variável composta homogênea unidimensional, ou seja, uma sequência finita de variáveis todas do mesmo tipo, com o mesmo identificador (mesmo nome) e alocadas sequencialmente na memória.

a	d	R	@	t	s	G	%	ç	p

Vetores

Um vetor é variável como qualquer outra existente no código. E, por isso, segue o mesmo padrão de comportamento. Podendo ser caracterizado por:

- Nome
- Endereço de memória
- Tipo
- Valor
- Escopo
- Tempo de vida

Declaração de vetores em Java

<Tipo de dados> [] <identificador>;

int [] x;

<Tipo de dados> <identificador>[];

float n[];

Todo vetor é um espaço linear na memória dividido em várias variáveis componentes de acordo com o tamanho declarado.

Inicialização de vetores em Java

A alocação de memória ocorre na inicialização do vetor (pode-se inicializar o vetor no momento de sua declaração).

```
int [] x = new int[4];

double [] y = {1.5, 3.6, 7.77};
```

Acesso aos elementos

Os componentes de um vetor são indexados sequencialmente começando em zero;

- Para identificar cada componente utilizamos o nome do vetor seguido da posição do componente na sequência entre colchetes;
- A posição de um componente é chamada de índice;

Exemplo:

```
float [] notas = new float[10];
notas[0] = 0.6f;
System.out.println(notas[0]);
```

Organização do código

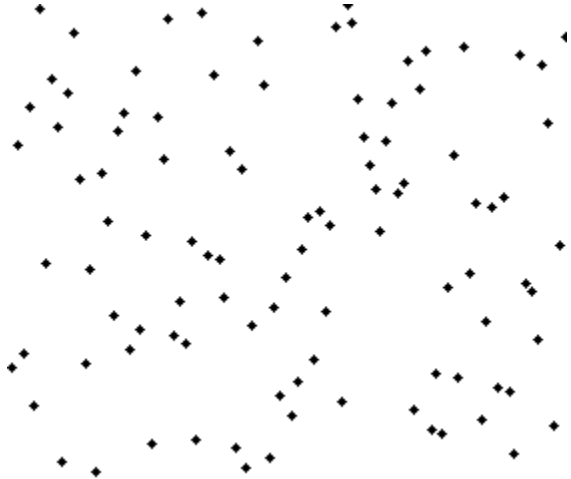
```java
import java.util.Random;
public class Maior_Menor{
Run | Debug
public static void main(String[] args){
int vetor []; //declaração do vetor
vetor = new int[50];//alocação de memória
Random numrandomico = new Random (); //declara variavel do tipo randomico
int maxValue = Integer.MIN_VALUE ; //atribui a variavel maxValue o menor valor de inteiro
int minValue = Integer.MAX_VALUE ; //atribui a variavel minValue o maior valor de inteiro

//preenche o vetor de 50 posições com numeros randomicos
for(int i = 0; i < 50; i++)
vetor[i]= numrandomico.nextInt (bound:999);

//descobrindo o menor e maior valor no vetor
for(int i = 0; i < 50; i++){
if(vetor[i] > maxValue)
maxValue = vetor[i];
if(vetor[i] < minValue)
minValue = vetor[i];
}
//exibe o maior e menor valor
System.out.println(("O maior valor é "+ maxValue));
System.out.println(("O menor valor é "+ minValue));

System.out.println(("Vetor randomico"));
//exibe todo o vetor
for(int num:vetor)
System.out.println(num);
}
}
```

```java
import java.util.Random;
public class Maior_Menor{
    Run | Debug
    public static void main(String[] args){
        int vetor []; //declaração do vetor
        vetor = new int[50];//alocação de memória
        Random numrandomico = new Random (); //declara variavel do tipo randomico
        int maxValue = Integer.MIN_VALUE ; //atribui a variavel maxValue o menor valor de inteiro
        int minValue = Integer.MAX_VALUE ; //atribui a variavel minValue o maior valor de inteiro
        //preenche o vetor de 50 posições com numeros randomicos
        for(int i = 0; i < 50; i++){
            vetor[i]= numrandomico.nextInt (bound:999);
        }

        //descobrindo o menor e maior valor no vetor
        for(int i = 0; i < 50; i++){
            if(vetor[i] > maxValue){
                maxValue = vetor[i];
            }

            if(vetor[i] < minValue){
                minValue = vetor[i];
            }
        }
        //exibe o maior e menor valor
        System.out.println(("O maior valor é "+ maxValue));
        System.out.println(("O menor valor é "+ minValue));

        System.out.println(("Vetor randomico"));
        //exibe todo o vetor
        for(int num:vetor){
            System.out.println(num);
        }
    }
}
```

Ordenação

Dado um vetor de N posições, preenchido com números inteiros aleatórios, construa um algoritmo que ordene seus elementos do menor para o maior.

```
//ordenação de vetor simples
for(int i =0; i <10; i++){
    for(int j =0; j <9; j++){
        if(vetor[j]> vetor[j+1]){
            aux = vetor[j];
            vetor[j] = vetor[j+1];
            vetor[j+1] = aux;
        }
    }
}
```



Busca Binária

Este método de pesquisa é muito mais rápido que a pesquisa sequencial, e usa como base que o vetor já está ordenado:
Dado um vetor: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

- Encontre a posição referente ao meio do vetor, e verifica se o valor procurado é o elemento do meio do vetor.
- Se encontrou o valor:
  Imprime uma mensagem de confirmação.
- Se não encontrou o valor:
  Se o valor procurado é menor que o valor que está no meio do vetor, a posição final do vetor será uma posição antes do meio do vetor.

  Se o valor procurado é maior que o valor que está no meio do vetor, a posição inicial do vetor será uma posição depois do meio do vetor.
- Volte para o passo 1.

Busca Binária

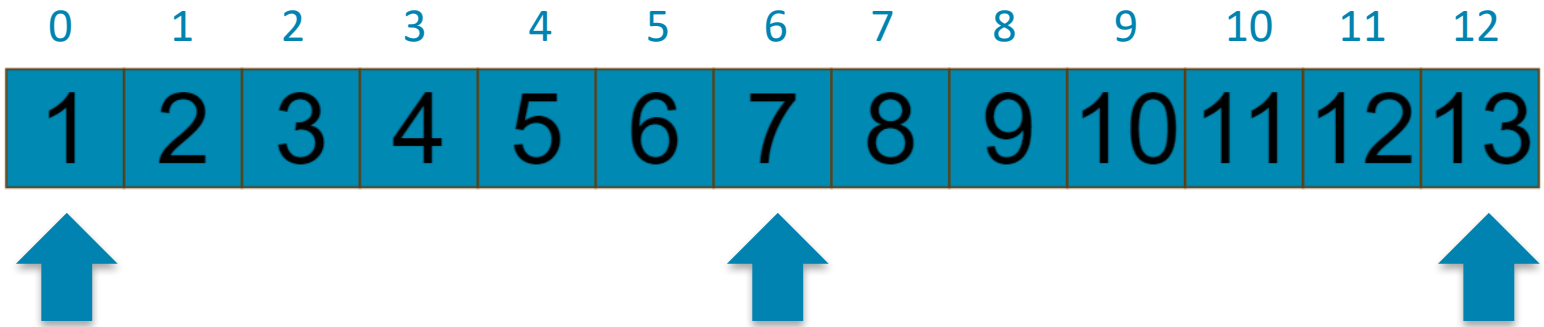
Pesquisa pelo valor 4.

O índice da posição inicial é 0 (zero).

O índice da posição final é 12.

O índice da posição meio é 6 (A posição do meio é igual ao (inicio + fim) / 2).

Verifica se o valor do vetor que está na posição do meio é igual ao valor procurado, neste caso o valor do meio do vetor é 7 e estamos procurando o valor 4.



Busca Binária

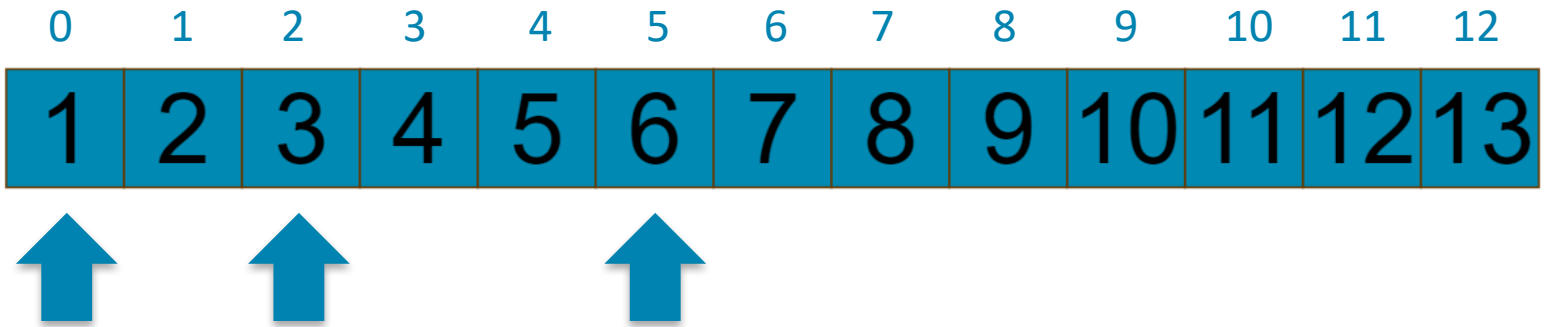
Pesquisa pelo valor 4.

O índice da posição inicial é 0 (zero).

O índice da posição final é 5.

O índice da posição meio é 2 (A posição do meio é igual ao (inicio + fim) / 2).

Verifica se o valor do vetor que está na posição do meio é igual ao valor procurado, neste caso o valor do meio do vetor é 3 e estamos procurando o valor 4.



Busca Binária

Pesquisa pelo valor 4.

O índice da posição inicial é 3.

O índice da posição final é 5.

O índice da posição meio é 3 (A posição do meio é igual ao (inicio + fim) / 2).

Verifica se o valor do vetor que está na posição do meio é igual ao valor procurado, neste caso o valor do meio do vetor é 4 e estamos procurando o valor 4.

0	1	2	3	4	5	6	7	8	9	10	11	12
1	2	3	4	5	6	7	8	9	10	11	12	13

Busca Binária

Pesquisa pelo valor 4.

O índice da posição inicial é 3.

O índice da posição final é 5.

O índice da posição meio é 4 (A posição do meio é igual ao (inicio + fim) / 2).

Verifica se o valor do vetor que está na posição do meio é igual ao valor procurado, neste caso o valor do meio do vetor é 5 e estamos procurando o valor 4.

0	1	2	3	4	5	6	7	8	9	10	11	12
1	2	3	4	5	6	7	8	9	10	11	12	13

Exercícios

1. Escreva um programa que leia dois vetores inteiros com dez posições cada. A partir desses vetores, carregue um terceiro vetor onde o valor de cada elemento será a média dos elementos de mesmo índice nos dois vetores anteriores.

2. Escreva um programa que carregue um vetor inteiro de cem posições com números aleatórios entre 0 e 100. Percorrendo o vetor criado apenas uma vez, imprima a posição onde ocorre o menor o valor, a soma dos números armazenados e preencha os valores de um novo vetor com metade do tamanho do vetor original onde a primeira posição do novo vetor é igual à soma da primeira e da última posição do vetor original. A segunda posição do novo vetor é a soma da segunda e da penúltima posição do vetor original e assim em diante.

3. Escreva um programa que carregue dois vetores inteiros com 5 posições, sendo um com números pares e o outro com números ímpares. O usuário pode digitar os números em qualquer sequência e o programa deverá armazená-los no vetor correto na ordem em que foram informados pelo usuário.

4. Escreva um programa que ordene um vetor de tamanho arbitrário preenchido com números aleatórios e execute a pesquisa por um valor passado como parâmetro utilizando o algoritmo da busca binária.

5. Implemente dois algoritmos que ordenem os elementos de um vetor em ordem crescente e compare o número de trocas que eles efetuam durante a ordenação de:

a) um vetor criado com os números de 1 até 100 aleatoriamente distribuídos.

b) um vetor criado com os números de 1 até 100 ordenados em ordem decrescente.